

Clase Teórica : Congestión

Tomás F. Melli

October 2025

Índice

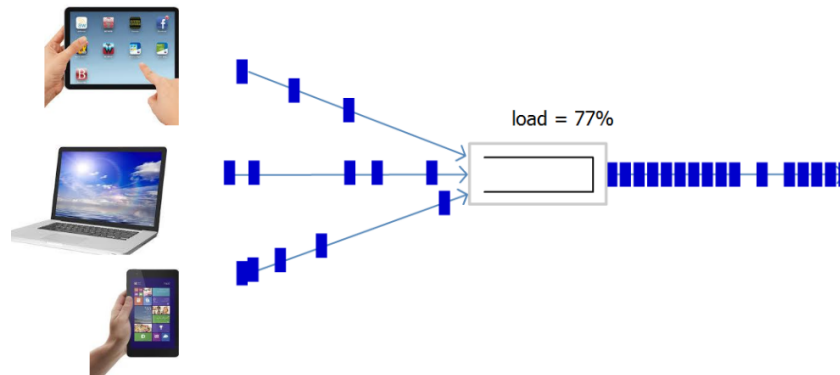
1	Introducción	3
1.1	Multiplexación Estadística	3
1.2	La Naturaleza de Internet	3
1.3	Administración de Buffers	3
1.4	Congestión	5
1.5	Perspectivas de la congestión	5
1.6	Soluciones y Estrategias	5
2	Análisis de Congestión: Caso de un Buffer	5
2.1	Teoría de Colas: El Sistema M/M/1	6
2.1.1	Esperanza de N y T	7
2.2	Delay promedio en M/M/1	7
2.3	Sistema M/M/m: Múltiples Servidores	8
3	Fundamentos del Control de la Congestión	8
3.1	Consideraciones sobre los Nodos	8
3.2	Control de Congestión vs. Control de Flujo	9
3.3	Métricas para Detectar Congestión	9
3.4	Políticas que influyen en la congestión	9
3.5	Causas de la Congestión	10
3.6	Control de Congestión	10
3.6.1	Implementación del Control de Congestión	11
3.6.2	Modelo de Servicio	11
3.7	Criterios de Evaluación del Control de Congestión	11
3.8	Eficiencia	11
3.9	Equidad	11
3.10	Desempeño en Función de la Carga	12
3.11	Congestión y Calidad de Servicio (QoS)	12
3.12	Resumen y Consideraciones Finales	12
4	Antecedentes y Teoría de Control Aplicada a la Congestión	12
4.1	Teoría de Control Aplicada a Redes	13
4.2	Taxonomía de Algoritmos de Control de Congestión	13
4.3	Control de Congestión vs. Congestión Preventiva	14
4.4	Realimentación Implícita vs. Explícita	14
5	RED (Random Early Detection)	15
5.1	Notificación implícita de congestión	15
5.2	Algoritmo RED	15
5.3	Cálculo de la probabilidad de descarte	16
5.4	Ajustes y recomendaciones	16
6	Flow Random Early Detection (FRED)	16
7	Políticas de Tráfico	17
7.1	Parámetros típicos	17
7.2	Algoritmos clásicos de Traffic Shaping	18

8	Impacto del Control de Congestión en TCP	18
8.1	Metodología de análisis macroscópico de TCP	19
8.2	Análisis de un ciclo	19
8.3	Probabilidad de pérdida y paquetes transmitidos	20
8.4	Sender Maximum Segment Size (SMSS) según RFC 2581	20

1 Introducción

La congestión en redes de computadoras es un fenómeno inherente a la naturaleza compartida de Internet y otras redes de datos. Se produce cuando la demanda de recursos —tales como enlaces de comunicación y buffers— se encuentra al límite o incluso excede la capacidad disponible. Este estado de sobrecarga sostenida afecta directamente la calidad del servicio (QoS), provocando retardos, pérdidas de paquetes y la necesidad de retransmisiones.

1.1 Multiplexación Estadística



Para comprender la congestión es necesario pensar en términos de distribuciones estadísticas de tráfico, dado que los patrones de envío y recepción de datos no son uniformes. La **multiplexación estadística** permite que múltiples flujos compartan un mismo canal físico, aprovechando que no todos los usuarios transmiten datos de manera simultánea.

Por ejemplo, si consideramos un buffer cuya ocupación promedio es del 90%, esto significa que, aunque en la mayoría del tiempo el buffer no está completamente lleno, existen momentos puntuales en que podría saturarse, generando retrasos o pérdidas de paquetes.

1.2 La Naturaleza de Internet

Internet es una red de recursos finitos que son compartidos entre múltiples usuarios y servicios. Esto implica que:

- Los buffers y enlaces tienen capacidad limitada.
- La agregación de diferentes flujos de tráfico puede llevar a que la demanda exceda los recursos disponibles.
- La diferencia entre la velocidad de acceso en la LAN y la velocidad disponible en la WAN puede generar cuellos de botella. Por ejemplo, múltiples conexiones de 100 Mbps en LAN pueden converger a un enlace WAN de solo 3 Mbps, provocando acumulación de paquetes en los buffers.

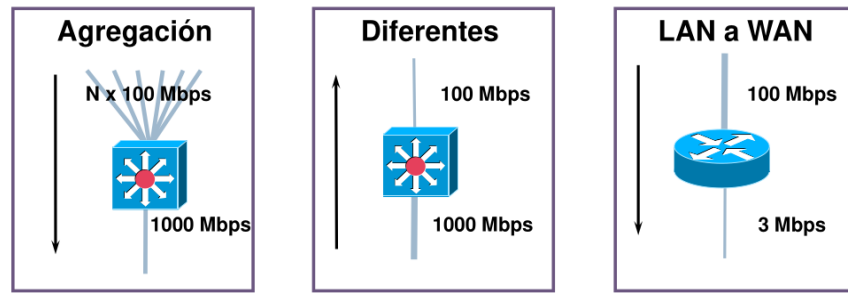
1.3 Administración de Buffers

Los buffers son estructuras diseñadas para almacenar temporalmente los datos antes de ser transmitidos. Su administración es crítica porque:

- **Buffering:** Reduce pérdidas de paquetes al almacenar datos temporalmente, pero incrementa el retardo total de la transmisión, calculado como:

$$\text{Delay} = T_{\text{tx}} + T_{\text{prop}} + T_{\text{queue}}$$

- **Buffer Overflow:** Cuando los buffers se saturan, los paquetes adicionales se descartan, provocando retransmisiones y degradación de QoS.
- Para garantizar un servicio de calidad, es necesario reservar espacio suficiente en los buffers o implementar estrategias de control que eviten su saturación.



Agregación

Este ejemplo muestra cómo se combinan varios flujos de tráfico en un mismo enlace:

- Supongamos que tenemos N LANs, cada una con un enlace de 100 Mbps .
- Cuando todas las LANs envían datos simultáneamente hacia un mismo punto (un router o switch), el tráfico total es:

$$\text{Tráfico agregado} = N \times 100 \text{ Mbps}$$

- Por ejemplo, si $N = 10$: $10 \times 100 \text{ Mbps} = 1000 \text{ Mbps}$.

La agregación puede superar la capacidad del enlace de salida, generando que los buffers comiencen a llenarse.

Diferentes

Este ejemplo resalta que no todos los enlaces tienen la misma capacidad:

- Imaginemos un enlace de agregación de 1000 Mbps que combina múltiples LANs.
- Cada LAN puede tener diferentes tasas de envío (100 Mbps , 50 Mbps , etc.).
- El router debe **manejar la suma de flujos distintos**, cada uno con características propias.

Aunque el promedio de uso del enlace pueda estar dentro de límites, picos de tráfico de ciertas LANs pueden saturar los buffers. Por eso es importante considerar la **distribución estadística** del tráfico, no solo la media.

LAN a WAN

Este ejemplo muestra el clásico **cuello de botella**:

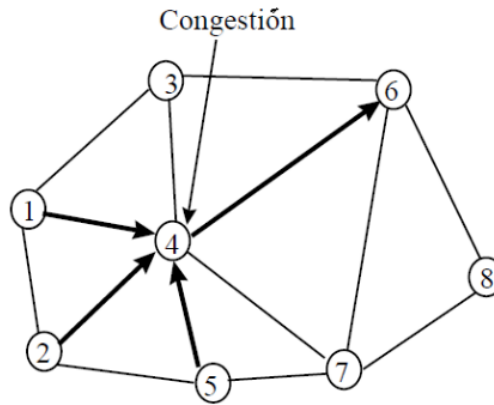
- Cada LAN puede enviar a 100 Mbps , pero el enlace WAN solo tiene 3 Mbps :

$$\text{Entrada LAN} = 100 \text{ Mbps}, \quad \text{Salida WAN} = 3 \text{ Mbps}$$

- El tráfico que llega al router supera ampliamente la capacidad de salida.
- Los paquetes se acumulan en los buffers del router.
- Si el buffer se llena, se producen pérdidas de paquetes $\rightarrow$ retransmisiones $\rightarrow$ degradación de QoS.
- Incluso antes de saturarse, los paquetes experimentan **delay de cola**, aumentando el retardo total:

$$\text{Delay} = T_{\text{tx}} + T_{\text{prop}} + T_{\text{queue}}$$

1.4 Congestión



Se define la congestión como un estado de sobrecarga sostenida de la red en el cual:

- La demanda de recursos se acerca o excede la capacidad disponible.
- Las consecuencias son perceptibles en términos de QoS degradada, incluyendo retardos aumentados, pérdida de paquetes y disminución en la eficiencia general de la red.

1.5 Perspectivas de la congestión

- **Desde un operador de red:** Un puerto de un CMTS (Cable Modem Termination System) se considera en estado de “casi congestión” si el tráfico que fluye hacia o desde ese puerto excede un nivel determinado (Port Utilization Threshold) durante un período específico (Port Utilization Duration). Este umbral se mide como un porcentaje del ancho de banda agregado del puerto en el período de tiempo considerado.
- **Desde un usuario:** La red se percibe como congestionada si la calidad del servicio experimentada disminuye debido al aumento de la carga de tráfico. Esto puede manifestarse en retardos, interrupciones de servicios o velocidades de transmisión inferiores a las esperadas.

1.6 Soluciones y Estrategias

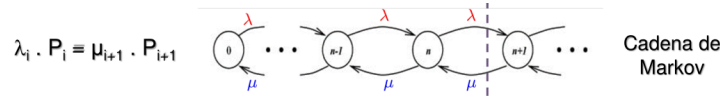
Para mitigar la congestión existen varias aproximaciones:

1. **Sobredimensionamiento (overprovisioning):** Incrementar la capacidad de enlaces y buffers para absorber picos de tráfico.
2. **Diseño cuidadoso:** Planificar la red para casos específicos de alta demanda.
3. **Control proactivo (preventivo):** Implementar políticas que eviten la congestión antes de que ocurra, en lugar de reaccionar después. Esto incluye:
 - Decrementar la carga de tráfico mediante regulación del uso.
 - Asignación de ancho de banda por parte de proveedores a usuarios específicos.
 - Incentivos para el uso uniforme de la red y penalizaciones para el uso excesivo.
 - Estrategias disuasivas, como retardos cognitivos en interfaces web, que limitan la generación de nuevas demandas de tráfico.

2 Análisis de Congestión: Caso de un Buffer

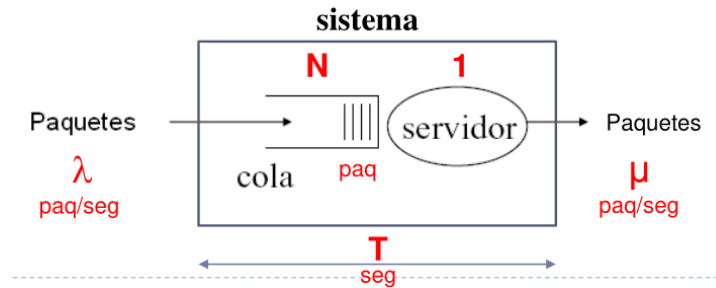
La congestión en redes puede analizarse formalmente mediante la teoría de colas, considerando los buffers como sistemas de almacenamiento temporal donde los paquetes esperan a ser procesados por un servidor (por ejemplo, un enlace o un router). Este análisis permite cuantificar el comportamiento de los paquetes en el sistema, el tiempo promedio de espera y la probabilidad de congestión bajo distintas condiciones de tráfico.

2.1 Teoría de Colas: El Sistema M/M/1



Un sistema **M/M/1** es un modelo clásico de teoría de colas que permite estudiar el comportamiento de un buffer con un único servidor bajo ciertas hipótesis:

- Las llegadas de paquetes siguen un proceso **Markoviano**, es decir, un proceso de Poisson, con tasa λ paquetes/seg.
- El tiempo de servicio de cada paquete también sigue una distribución **exponencial** con tasa μ paquetes/seg.
- Hay un único servidor que procesa los paquetes.
- La cola tiene capacidad infinita, por lo que no se descarta ningún paquete mientras esté en el sistema.



- **Sistema:** Representa el buffer o cola donde los paquetes esperan para ser procesados.
- **Llegadas (λ):** Tasa de entrada de paquetes al sistema, medida en paquetes por segundo.
- **Servicio (μ):** Tasa a la que el servidor procesa los paquetes, también en paquetes por segundo.
- **Tiempo de estadía (T):** Tiempo promedio que cada paquete permanece dentro del sistema, incluyendo espera en la cola y procesamiento (segundos).

Notación

- N : número total de paquetes en el sistema (cola + servidor).
- T : tiempo promedio que un paquete permanece en el sistema (segundos).
- λ : tasa de llegada de paquetes al sistema (paquetes/seg).
- μ : tasa de servicio del servidor (paquetes/seg).
- $1/\lambda$: tiempo promedio entre llegadas.
- $1/\mu$: tiempo promedio de servicio.
- $\rho = \lambda/\mu$: intensidad del sistema.

Para que el sistema alcance un **estado estacionario** es necesario que $\rho < 1$, es decir, que la tasa de llegada sea menor que la tasa de servicio.

En términos de cadenas de Markov, la probabilidad de que haya N paquetes en el sistema se denota P_N y cumple:

$$\lambda_i \cdot P_i = \mu_{i+1} \cdot P_{i+1}$$

Si $\rho > 1$, la tasa de llegada excede la capacidad de servicio y el buffer se desborda, generando congestión.

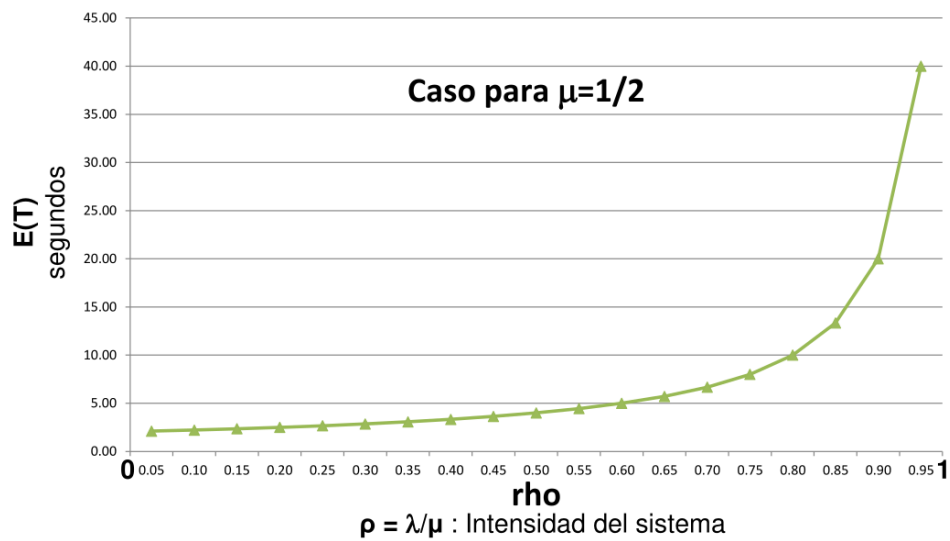
2.1.1 Esperanza de N y T

El número esperado de paquetes en el sistema $E(N)$ y el tiempo promedio de estadía $E(T)$ se relacionan mediante la ley de Little:

$$\begin{aligned} E(N) &= \frac{\rho}{1 - \rho} \\ E(N) &= \lambda \cdot E(T) \\ \Rightarrow E(T) &= \frac{E(N)}{\lambda} \\ &= \frac{\rho}{\lambda(1 - \rho)} \\ &= \frac{\lambda/\mu}{\lambda(1 - \lambda/\mu)} \\ &= \frac{1}{\mu - \lambda} \end{aligned}$$

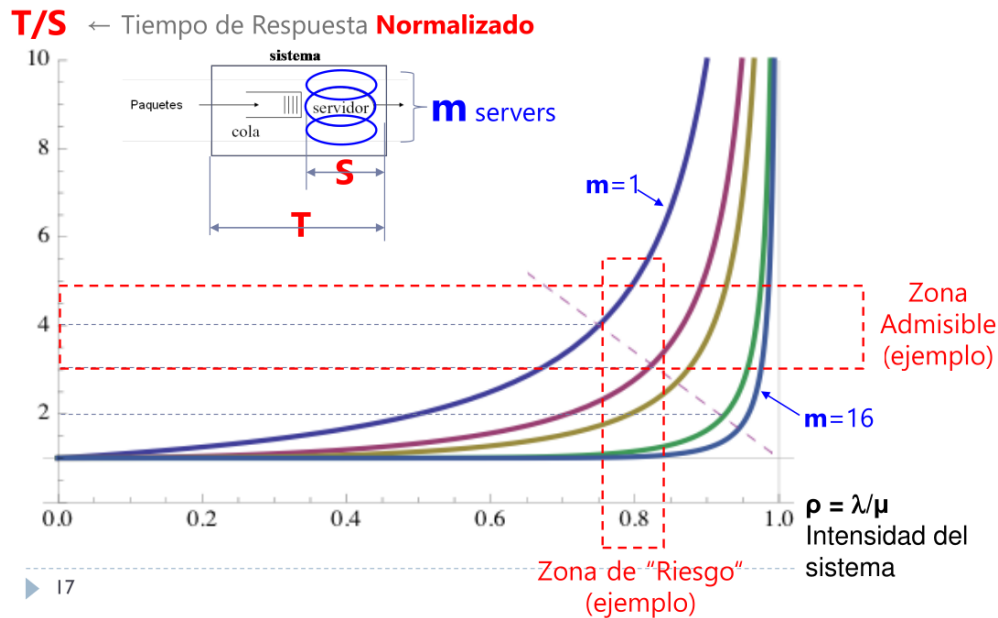
Estas expresiones permiten cuantificar cómo el incremento en la intensidad del sistema ρ afecta directamente tanto el número de paquetes acumulados en la cola como el tiempo de espera promedio.

2.2 Delay promedio en M/M/1



El **delay promedio** $E(T)$ aumenta drásticamente cuando la intensidad del sistema se aproxima a 1. Por ejemplo, para un servidor con $\mu = 1/2$ paquetes/seg, se observa que $E(T)$ crece de manera casi vertical a medida que $\rho \rightarrow 1$. Esto refleja la zona de riesgo de congestión, donde pequeños incrementos en la tasa de llegada generan grandes retrasos.

2.3 Sistema M/M/m: Múltiples Servidores



Para mejorar la capacidad de procesamiento y reducir el riesgo de congestión, se puede considerar un sistema con m servidores paralelos (M/M/m).

- Cada servidor procesa a tasa μ paquetes/seg.
- La intensidad del sistema se normaliza como $\rho = \lambda/(m\mu)$.
- El tiempo de respuesta promedio disminuye al incrementar el número de servidores m , desplazando el sistema hacia la **zona admisible** de operación.

Gráficamente, para distintos valores de m (1, 2, 3, 9, 16), se observa que el tiempo de respuesta normalizado T/S disminuye con más servidores, mientras que la zona de riesgo de congestión se desplaza hacia valores mayores de intensidad.

3 Fundamentos del Control de la Congestión

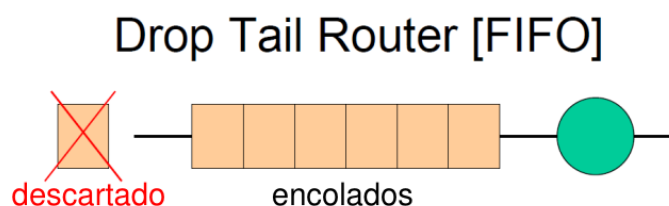
La congestión en redes se puede definir informalmente como la situación en la que *demasiadas fuentes utilizan una red compartida enviando demasiados datos demasiado rápido como para poder ofrecer una buena calidad de servicio (QoS)*.

Síntomas Típicos de Congestión

Los síntomas más comunes que indican que una red está congestionada incluyen:

- **Pérdida de paquetes:** Los buffers de routers o switches se saturan y se descartan paquetes.
- **Retardos crecientes:** Los paquetes experimentan mayor tiempo de espera debido a la encolación en los buffers.
- **Degradación de QoS:** Retrasos y pérdidas afectan la eficiencia del sistema y la experiencia del usuario.

3.1 Consideraciones sobre los Nodos



Algunos aspectos importantes a tener en cuenta sobre el manejo de paquetes en los nodos de la red:

- **FIFO:** Los paquetes se transmiten en el orden en que llegan, es un mecanismo de agendamiento (*scheduling*).
- **Drop Tail:** Los paquetes se descartan cuando se llena la capacidad del buffer, es una política de manejo de buffers.
- La combinación de FIFO y Drop Tail puede generar una **sincronización global** cuando los paquetes descartados provienen de diferentes conexiones no sincronizadas.

3.2 Control de Congestión vs. Control de Flujo

- **Control de Congestión:** Busca evaluar la capacidad de una subred completa para transportar un determinado tráfico agregado. La congestión es un efecto global que involucra a todos los hosts y routers compartiendo la red.
- **Control de Flujo:** Regula el tráfico punto a punto entre un transmisor y un receptor particular, evitando que un receptor lento sea sobrecargado por un transmisor rápido.

3.3 Métricas para Detectar Congestión

Varias métricas pueden indicar la presencia de congestión en la red:

- Porcentaje de paquetes descartados por falta de espacio en los buffers.
- Longitud media de una cola (buffer).
- Número de paquetes que generan *time out* y son retransmitidos.
- Retardo promedio de los paquetes (*average packet delay*).
- Desviación estándar del retardo de los paquetes (*standard deviation of packet delay*).

El crecimiento de una o varias de estas métricas indica congestión.

3.4 Políticas que influyen en la congestión

La congestión en redes no solo depende de la cantidad de tráfico, sino también de las políticas implementadas en cada capa del modelo de comunicación. Estas políticas determinan cómo se manejan los paquetes, los errores y los recursos de la red, y por lo tanto afectan directamente la aparición y mitigación de congestión.

Capa de Transporte

En la capa de transporte, las políticas que influyen en la congestión incluyen:

- **Retransmisión:** Cuando se detecta la pérdida de un paquete (por timeout o por ACK negativo), el transmisor reenvía el paquete. Retransmisiones excesivas pueden agravar la congestión si la red ya está saturada.
- **Almacenamiento en *cache* y reensamblado fuera de orden (*out-of-order*):** Los paquetes pueden llegar fuera de orden; mantenerlos en buffer permite reconstruir el flujo original sin solicitar retransmisión inmediata.
- **Confirmaciones de recepción (ACKs):** Sirven para informar al transmisor qué paquetes han sido recibidos correctamente. La política de cuándo y cómo enviar ACKs puede influir en la generación de tráfico adicional.
- **Control de flujo:** Limita la cantidad de datos que un transmisor puede enviar antes de recibir confirmación, evitando que el receptor se vea sobrecargado.

Capa de Red

En la capa de red, la congestión depende de cómo se gestionan los recursos y se transmiten los paquetes:

- **Circuitos virtuales vs. Datagramas:**
 - En circuitos virtuales, se reserva un camino y recursos a lo largo de toda la ruta, permitiendo cierta garantía de QoS, pero si el circuito se colapsa, puede afectar a todos los flujos sobre él.
 - En redes basadas en datagramas, los paquetes se enrutan de manera independiente, lo que puede provocar colisiones en nodos congestionados, pero permite más flexibilidad en la distribución del tráfico.
- **Encolamiento y políticas de descarte de paquetes:**

- FIFO, Drop Tail, Random Early Detection (RED), entre otros, determinan qué paquetes se transmiten primero y cuáles se descartan en caso de saturación.

- **Algoritmos de ruteo:**

- La elección de rutas alternativas y la actualización dinámica de tablas de ruteo permiten distribuir el tráfico y reducir la congestión en enlaces saturados.

Capa de Enlace

En la capa de enlace, las políticas que afectan la congestión se centran en la confiabilidad de la transmisión sobre enlaces físicos:

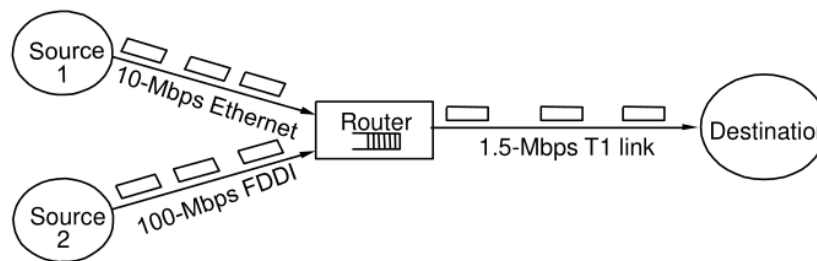
- **Retransmisión:** Protocolos de enlace como ARQ (Automatic Repeat reQuest) reenvían tramas perdidas o con errores, lo que puede incrementar la carga de la red si el canal tiene alta tasa de errores.
- **Almacenamiento en *cache*:** Permite mantener temporalmente tramas recibidas fuera de orden, para reensamblarlas correctamente sin necesidad de retransmisión inmediata.
- **Confirmación de recepción:** Indicadores de recepción correcta de tramas que evitan pérdidas de información, pero generan tráfico adicional (ACKs).
- **Control de flujo:** Evita que el transmisor sature el buffer del receptor en enlaces punto a punto, reduciendo así la probabilidad de congestión en el enlace.

3.5 Causas de la Congestión

Las principales causas que pueden provocar congestión incluyen:

- Sobrecarga de tráfico hacia una misma línea de salida (buffer lleno $\rightarrow$ *tail drop*).
- Incrementar la memoria no necesariamente resuelve el problema.
- Procesadores lentos o problemas en el software de ruteo.
- Cuellos de botella en algún punto del sistema (*links* rápidos hacia uno lento).
- La congestión tiende a autoalimentarse, empeorando la situación; en el límite puede generarse un **colapso de congestión** donde el *goodput* tiende a cero.

3.6 Control de Congestión



El control de congestión consiste en los esfuerzos realizados por los nodos de la red para prevenir o responder a sobrecargas que conduzcan a pérdidas no controladas de paquetes. Dos enfoques posibles:

- **Pre-asignación de recursos:** Reservar ancho de banda y espacio de buffers en routers y switches para evitar congestión. Esto puede causar subutilización si los recursos no se usan.
- **Control reactivo:** Liberar recursos y controlar la congestión solo cuando ocurre, distribuyendo los efectos entre todos los usuarios de manera equitativa.

3.6.1 Implementación del Control de Congestión

El control de congestión puede implementarse en dos puntos principales:

- **Hosts en los extremos de la red:** A través del protocolo de transporte, por ejemplo mediante ventana deslizante (*sliding window*).
- **Routers internos de la red:** Con políticas de encolado como *Random Early Detection (RED)* y otras disciplinas de *queueing*.

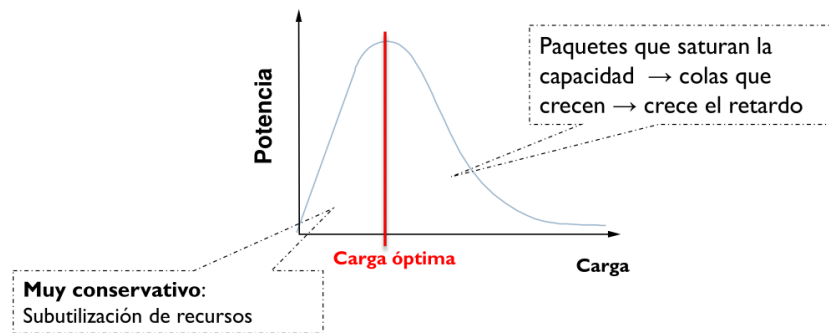
3.6.2 Modelo de Servicio

- Internet se asume por ahora como un modelo **best-effort** (mejor esfuerzo), en el que los paquetes se transmiten sin garantía estricta de QoS.
- Existen también múltiples niveles de QoS que permiten diferenciar servicios, por ejemplo:
 - Ancho de banda garantizado para *video streaming*.
 - Retardo reducido para *Voz sobre IP (VoIP)*.

3.7 Criterios de Evaluación del Control de Congestión

El control de congestión busca que la red sea utilizada de manera **eficiente** y al mismo tiempo **equitativa**. Para evaluar el desempeño de la red se utilizan varios criterios, que incluyen eficiencia, equidad y desempeño en función de la carga.

3.8 Eficiencia



Un buen indicador de eficiencia es la llamada **potencia**, definida como la relación entre el throughput y el delay:

$$\text{Potencia} = \frac{\text{throughput}}{\text{delay}}$$

- Si se generan demasiados paquetes que saturan la capacidad de la red, las colas en los routers crecen, incrementando el retardo.
- La **carga óptima** es aquella que maximiza la potencia, evitando saturaciones innecesarias.
- Un enfoque muy conservativo puede evitar la congestión, pero provoca subutilización de los recursos.

3.9 Equidad

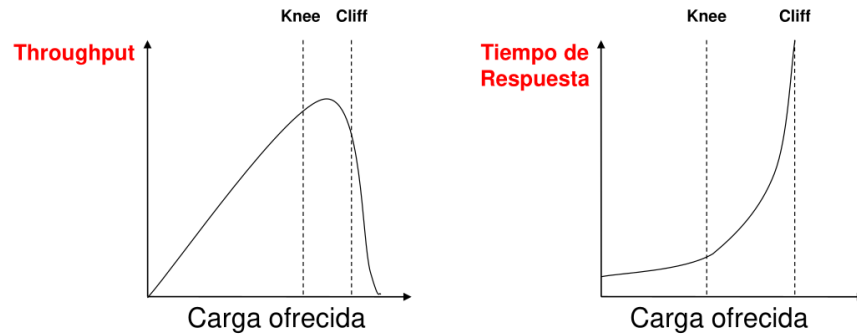
La equidad busca que los recursos de la red se compartan de manera equitativa entre los usuarios, tanto bajo condiciones normales como en situaciones de congestión.

- Un indicador común de equidad es el **Índice de Jain**, definido para n flujos con throughputs $(x_1, x_2, \dots, x_n)$ como:

$$f(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}, \quad \frac{1}{n} \leq f \leq 1$$

- Donde $f = 1$ representa equidad máxima, y valores menores indican desigualdad en la asignación de recursos.

3.10 Desempeño en Función de la Carga



El comportamiento de la red frente a distintos niveles de carga se puede describir mediante las curvas **knee** y **cliff**:

- A medida que la carga ofrecida a la red aumenta, el throughput crece casi linealmente.
- Cuando la carga se acerca a la capacidad de la red, los buffers de los routers comienzan a llenarse, aumentando el tiempo de respuesta y limitando el incremento del throughput.
- Si los buffers se sobrecargan, se producen pérdidas de paquetes, y la probabilidad de pérdida aumenta con incrementos adicionales de carga.
- Bajo cargas extremas, el tiempo de respuesta tiende a infinito y el throughput a cero; este punto se conoce como **cliff** debido a la caída extrema del throughput.

3.11 Congestión y Calidad de Servicio (QoS)

Proporcionar QoS es sencillo si la red nunca se congestiona; sin embargo, sobredimensionar todos los enlaces no siempre es posible ni deseable. Para garantizar QoS en presencia de congestión:

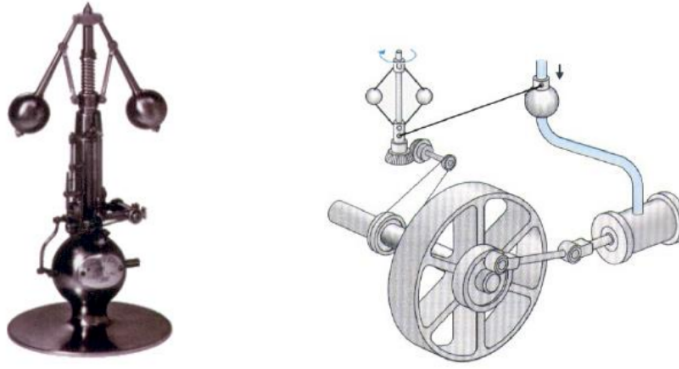
- Se requieren mecanismos que permitan diferenciar ciertos tipos de tráfico preferencial, cumpliendo así los **SLA (Service Level Agreements)**.
- Un SLA es típicamente estático y definido durante la negociación del contrato con el proveedor de servicios o ISP.

3.12 Resumen y Consideraciones Finales

- El término **control de congestión** describe los esfuerzos que debe realizar un nodo de red (router o end-host) para prevenir o responder a condiciones de sobrecarga.
- Alcanzar el punto de congestión es generalmente un mal síntoma, ya que la recuperación no siempre es posible.
- Es recomendable tomar medidas preventivas en lugar de correctivas cuando ya se ha detectado congestión.
- Una posible solución consiste en persuadir a algunos hosts para que disminuyan su tráfico, mejorando la situación general.
- Sin embargo, esto requiere señalización a hosts específicos; los mecanismos de control de congestión deben incorporar nociones de **alocación equitativa de recursos** para distribuir los efectos de manera justa entre todos los usuarios.

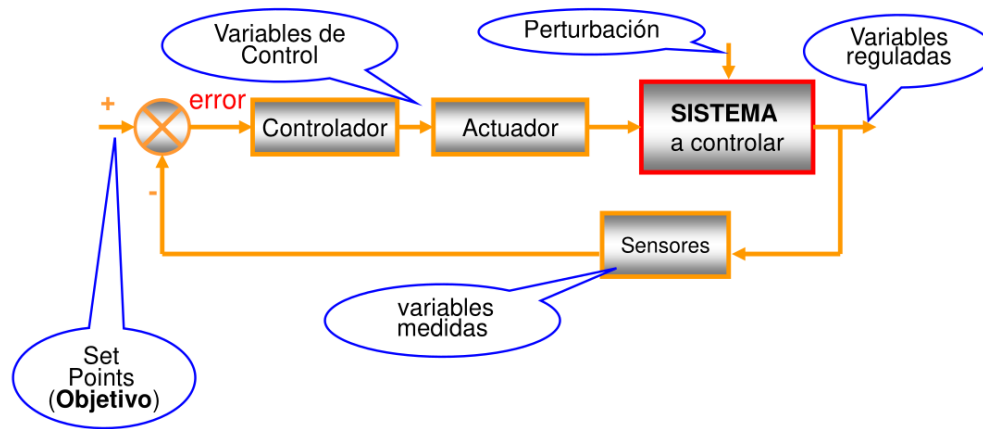
4 Antecedentes y Teoría de Control Aplicada a la Congestión

El concepto de control de sistemas se remonta a los primeros intentos de automatización industrial. Un ejemplo clásico es el **dispositivo de control de presión** desarrollado por **James Watt** en una máquina de vapor alrededor de 1760, cuyo objetivo era mantener la presión dentro de límites seguros mediante un mecanismo de retroalimentación que ajustaba automáticamente la válvula de vapor según la presión medida:



Estos principios históricos de control regulatorio inspiran los métodos modernos de **control de congestión en redes de computadoras**.

4.1 Teoría de Control Aplicada a Redes



En términos generales, un sistema de control se compone de los siguientes elementos:

- **Perturbación:** Cualquier factor externo que afecte el sistema.
- **Variables reguladas:** Las magnitudes que se desean mantener dentro de valores objetivos (*set points*).
- **Sensores:** Miden las variables del sistema.
- **Controlador:** Calcula la acción de control a partir de la diferencia (*error*) entre la referencia y la variable medida.
- **Actuador:** Aplica la acción de control sobre el sistema a controlar.

Este modelo se adapta al contexto de redes, donde los paquetes, buffers y enlaces representan las variables del sistema, y los algoritmos de control de congestión actúan como controladores que ajustan la tasa de envío de datos.

4.2 Taxonomía de Algoritmos de Control de Congestión

Según **Yang y Reddy (1995)**, los algoritmos de control de congestión se pueden clasificar en:

- **Lazo abierto:** Utilizados principalmente en redes de conmutación de circuitos (por ejemplo, GMPLS). La fuente no recibe retroalimentación del estado de la red.
- **Lazo cerrado:** Utilizados principalmente en redes de paquetes y basados en realimentación. Se dividen en:
 - **Realimentación implícita:** La fuente infiere la congestión a partir de eventos de la red, como pérdida de paquetes, *timeouts* o ACKs duplicados. Ejemplos: TCP Tahoe, TCP Reno, TCP Vegas, SCTP, TCP CUBIC. Normalmente implementada a nivel de transporte y relativamente simple de aplicar.
 - **Realimentación explícita:** Los nodos de la red proporcionan indicaciones directas de congestión a las fuentes mediante *packet marking* o señalización. Ejemplos: DECbit, ECN (Explicit Congestion Notification), ATM ABR. Requiere cooperación explícita entre fuentes y componentes de la red y cambios en los algoritmos de los nodos.

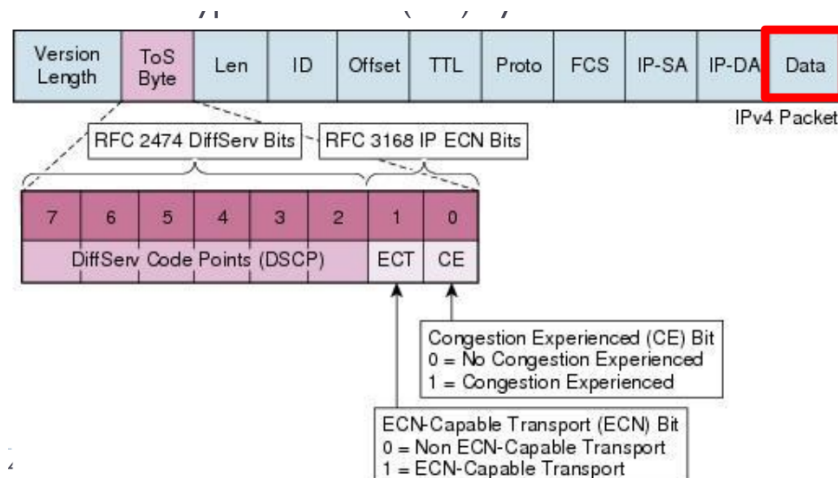
4.3 Control de Congestión vs. Congestión Preventiva

- **Congestion Control (reactivo):** El sistema responde a la congestión cuando ya se ha producido, ajustando la tasa de envío de datos para estabilizar la red.
- **Congestion Avoidance (preventivo o proactivo):** El sistema actúa antes de que ocurra la congestión, limitando la carga de tráfico o distribuyéndola de manera óptima para evitar saturaciones.

4.4 Realimentación Implícita vs. Explícita

Los algoritmos de control de congestión pueden diferenciarse según cómo la fuente recibe información sobre el estado de la red:

- **Realimentación implícita (*Implicit Feedback Congestion Control*):**
 - La red no envía información directa; la fuente infiere congestión a partir de eventos de la red.
 - Señales típicas incluyen:
 - * Paquetes descartados cuando la congestión se aproxima.
 - * *Timeouts* y ACKs duplicados.
 - Ejemplo: TCP End-to-End Congestion Control (TCP Tahoe, Reno, Vegas, SCTP, TCP CUBIC).
 - Ventajas: Implementación relativamente simple.
 - Normalmente implementado a nivel de transporte.
- **Realimentación explícita (*Explicit Feedback Congestion Control*):**
 - Los nodos de la red (routers, switches, software de red) envían señales directas de congestión a las fuentes.
 - Utiliza técnicas como *packet marking*.
 - Ejemplos:
 - * DEChit
 - * ECN (Explicit Congestion Notification)
 - * ATM ABR Congestion Control
 - * ICMP Source Quench
 - Proporciona información más precisa sobre la congestión.
 - Implementación más compleja, requiere cambios en la fuente y en los nodos de la red.
 - Requiere cooperación explícita entre las fuentes y los componentes de la red.
 - Ejemplo específico: IP ECN, que utiliza el byte *Type of Service (ToS)* en el encabezado IP para notificar congestión:



5 RED (Random Early Detection)

El algoritmo **RED** (Random Early Detection) tiene como objetivo principal **evitar la congestión** manteniendo el tamaño promedio de las colas en los routers en niveles relativamente bajos.

- RED es un método **implícito de Congestion Avoidance**.
- No requiere que los routers mantengan información sobre el estado de cada conexión.
- Fue diseñado para trabajar en colaboración con mecanismos de control de congestión en la capa de transporte, típicamente TCP.
- RED pertenece a la familia de técnicas conocidas como **AQM** (Active Queue Management), a veces denominadas **AQM-RED**.

5.1 Notificación implícita de congestión

RED proporciona una forma de notificación implícita sobre la inminencia de congestión:

- Cuando se detecta congestión, simplemente se descarta el paquete entrante (en TCP esto genera *timeouts* que reducen la tasa de envío).
- Alternativamente, se podría marcar el paquete explícitamente usando ECN.
- RED realiza un **descarte aleatorio temprano** para evitar que la cola se llene completamente.

En lugar de esperar a que el buffer se llene, cada paquete entrante puede ser descartado con cierta probabilidad cuando la cola excede un nivel de llenado determinado.

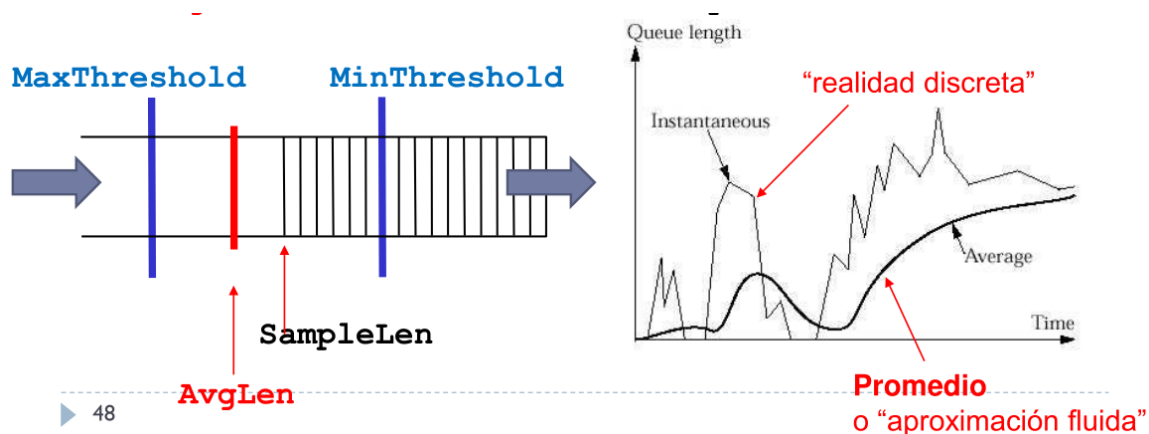
5.2 Algoritmo RED

RED calcula un **largo promedio de la cola** (*AvgLen*) mediante un suavizado exponencial:

$$\text{AvgLen} = (1 - \text{Weight}) \cdot \text{AvgLen} + \text{Weight} \cdot \text{SampleLen}$$

- $0 < \text{Weight} < 1$.
- *SampleLen* es el tamaño instantáneo de la cola, actualizado cada vez que llega o sale un paquete.
- *AvgLen* representa una versión suavizada de *SampleLen* o una aproximación fluida del tamaño de la cola.

RED utiliza dos umbrales de tamaño promedio de la cola:



- **MinThreshold:** Si $\text{AvgLen} \leq \text{MinThreshold}$, el paquete se encola normalmente.
- Si $\text{MinThreshold} < \text{AvgLen} < \text{MaxThreshold}$, se calcula una probabilidad P de descartar el paquete entrante.
- **MaxThreshold:** Si $\text{AvgLen} \geq \text{MaxThreshold}$, se descarta el paquete entrante de manera segura.

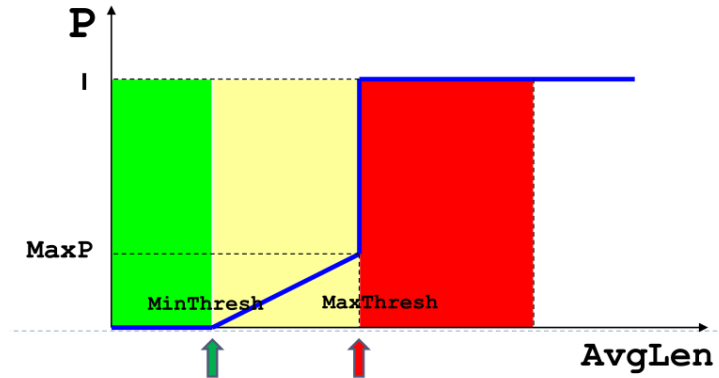
5.3 Cálculo de la probabilidad de descarte

La probabilidad P de descartar un paquete cuando la cola está entre los dos umbrales se calcula así:

$$\text{TempP} = \text{MaxP} \cdot \frac{\text{AvgLen} - \text{MinThreshold}}{\text{MaxThreshold} - \text{MinThreshold}},$$
$$P = \frac{\text{TempP}}{1 - \text{count} \cdot \text{TempP}}$$

- count es el número de paquetes encolados mientras AvgLen está entre los dos umbrales.

Esto genera una curva de probabilidad creciente de descarte a medida que la cola se acerca a MaxThreshold :



5.4 Ajustes y recomendaciones

- La probabilidad de descartar un paquete de un flujo particular es aproximadamente proporcional a la porción de ancho de banda que el flujo está utilizando.
- MaxP típicamente se fija en 0.02.
- Si el tráfico es *rafagoso*:
 - MinThreshold debe ser suficientemente grande para mantener la utilización del enlace en un nivel aceptable.
 - La diferencia entre MaxThreshold y MinThreshold debe ser mayor que el incremento típico en el largo promedio de la cola calculado en un RTT.
- Como regla práctica para Internet, se suele fijar $\text{MaxThreshold} \approx 2 \cdot \text{MinThreshold}$. Esto implica que, bajo condiciones normales, aproximadamente uno de cada 50 paquetes podría ser descartado.

6 Flow Random Early Detection (FRED)

FRED (Flow Random Early Detection) es una evolución del algoritmo RED que busca mejorar la **imparcialidad** (fairness) en la gestión de congestión.

- Una de las características principales de RED es que proporciona cierta imparcialidad, descartando paquetes de una conexión según la parte de ancho de banda que ocupa.
- Sin embargo, RED presenta problemas de **fairness**, especialmente con conexiones de baja velocidad.
- En RED, cuando se alcanza el umbral máximo, los paquetes se descartan aleatoriamente. Esto puede provocar que un paquete de un flujo que utiliza pocos recursos sea descartado mientras otros flujos más agresivos continúan utilizando más ancho de banda.
- Cuando la longitud media de la cola se encuentra entre los dos umbrales, todos los paquetes entrantes se descartan con la misma probabilidad, sin distinguir entre flujos.

Problemas de usuarios acaparadores

- RED no protege efectivamente contra usuarios que intentan ocupar todo el ancho de banda (*bandwidth hogs*).
- Aunque se cumple el umbral máximo, si un usuario no disminuye su tasa de envío, RED no posee mecanismos para proteger a los demás usuarios.
- Como consecuencia, usuarios egoístas podrían terminar monopolizando el enlace y degradando el servicio para otros.

Solución de FRED

FRED introduce mejoras para garantizar mayor **imparcialidad** entre flujos activos:

- Mantiene **umbrales y ocupaciones del buffer** por cada flujo activo, usando información por conexión.
- Necesita almacenar información para cada flujo, lo que incrementa el costo operativo en los routers.
- Debe identificar cada flujo con paquetes en el buffer y actualizar la información por cada paquete recibido.
- Para ello, FRED analiza la fuente, la dirección de destino, los puertos y la identificación del protocolo de cada paquete.
- Esto permite que el descarte de paquetes se realice de manera proporcional al uso real de cada flujo, evitando que usuarios egoístas acaparen recursos.

7 Políticas de Tráfico

El tráfico en redes suele ser **rafagoso** (*bursty*), lo que impacta directamente en la congestión. Para mitigar este efecto, se utilizan técnicas de **Traffic Shaping**, que buscan regular la transmisión de paquetes de manera más predecible.

- **Traffic Shaping** es un método de lazo abierto que guía la congestión, forzando a los paquetes a transmitirse a una velocidad más constante.
- Su objetivo es mantener el tráfico *constante*, regulando la tasa de transmisión media y su desviación estándar (reduciendo el *burstiness* de los datos).
- En redes de *circuito virtual* (CV), durante el *setup* del circuito, el usuario y el proveedor acuerdan el perfil de tráfico (*shaping* del circuito).
- Esto requiere un acuerdo explícito entre proveedor y cliente.

7.1 Parámetros típicos

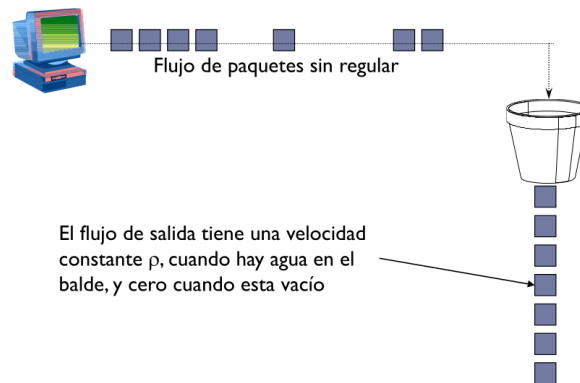
Algunos parámetros clave para Traffic Shaping incluyen:

- **Average Rate:** Tasa media de transmisión, por ejemplo 100 paquetes por segundo o 6000 paquetes por minuto.
- **Peak Rate:** Tasa máxima permitida, por ejemplo 1500 paquetes por segundo con un promedio de 6000 ppm.
- **Burst Size:** Número máximo de paquetes enviados consecutivamente en un período corto de tiempo.
- La longitud del intervalo de medición es un aspecto crucial para el desempeño del *shaping*.

7.2 Algoritmos clásicos de Traffic Shaping

Entre los algoritmos más conocidos para control de QoS se encuentran:

- **Leaky Bucket:**



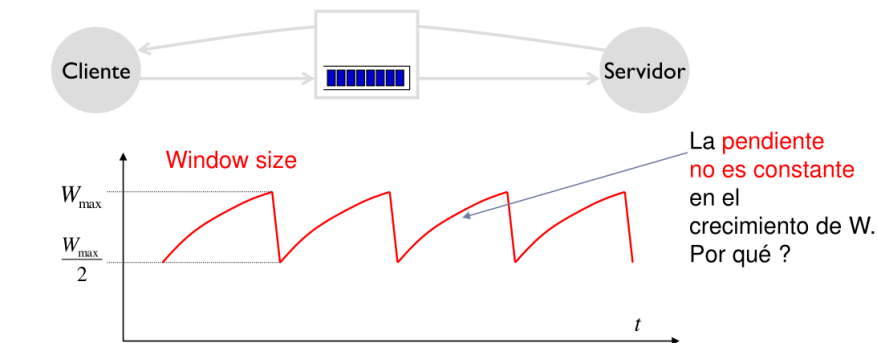
- El flujo de salida tiene velocidad constante ρ mientras hay "agua" en el balde, y cero cuando está vacío.
- Se moderan las ráfagas, estableciendo un límite estricto de throughput.

- **Token Bucket:**

- Permite picos de tráfico y ráfagas, a diferencia del Leaky Bucket que fuerza un patrón rígido.
- Mantiene *tokens* generados a un rate de 1 token cada ΔT segundos.
- Para transmitir un paquete, se consume un token. Si no hay tokens disponibles, el paquete espera o se descarta.
- Hosts inactivos pueden "ahorrar" tokens para enviar ráfagas posteriores.
- Permite que proveedores de servicios, como cablemódems, autoricen temporalmente picos de tráfico superiores al contratado.

8 Impacto del Control de Congestión en TCP

El control de congestión en TCP busca ajustar dinámicamente la **ventana de transmisión** (window size, W) para evitar saturar la red, mientras maximiza el throughput.



La idea básica por cada *Round-Trip Time* (RTT) es:

- **Si se recibe un ACK:**

$$W \leftarrow W + \frac{1}{W} \quad (\text{Additive Increase})$$

- **Si se pierde un ACK:**

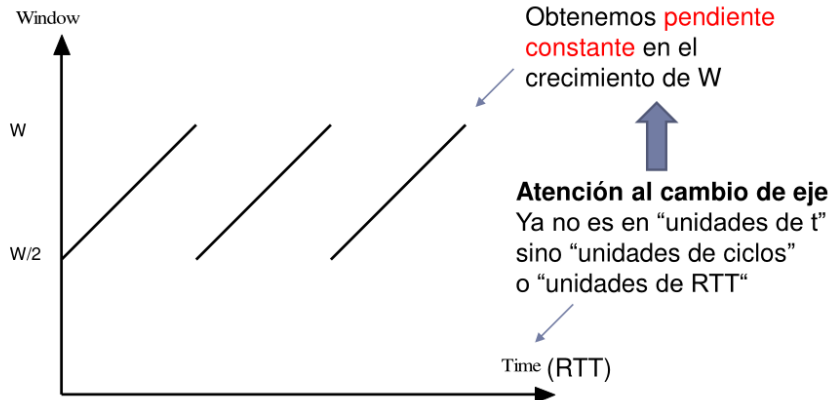
$$W \leftarrow \frac{W}{2} \quad (\text{Multiplicative Decrease})$$

- Esta estrategia permite un crecimiento gradual de la ventana mientras la red está libre de congestión y una reducción rápida ante detección de pérdida.
- La pendiente de crecimiento de W no es constante en unidades de tiempo, sino en unidades de ciclos o RTT.

8.1 Metodología de análisis macroscópico de TCP

Siguiendo Mathis et al., se estudia el comportamiento de TCP bajo condiciones de *steady state* con las siguientes suposiciones:

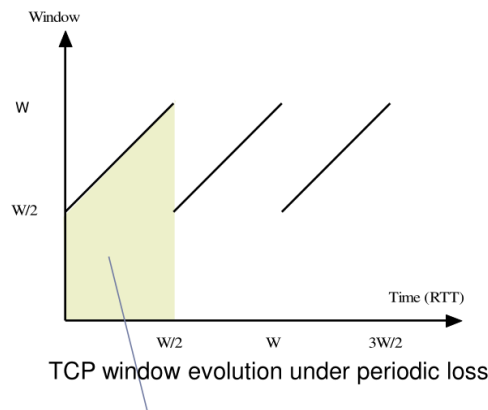
- RTT constante durante la conexión.
- Cada paquete es descartado con probabilidad p .
- El enlace tiene ancho de banda suficiente para soportar la conexión.
- La señal de congestión es periódica: se transmiten $1/p$ paquetes con éxito seguidos de un paquete descartado.
- Se analiza el régimen permanente de una conexión de larga duración.



- Sea W el máximo valor de la ventana alcanzado en un ciclo.
- Se asume la versión de *Congestion Avoidance* donde la ventana mínima tras una pérdida es $W/2$.
- Durante un ciclo de transmisión, la ventana evoluciona linealmente desde $W/2$ hasta W .

Window (paquetes)	Time (RTT)
$W/2$	inicio del ciclo
W	final del ciclo

8.2 Análisis de un ciclo



- El número de paquetes transmitidos por ciclo se calcula como el área bajo la evolución de la ventana:

$$\text{pkts TX/ciclo} = \left(\frac{W}{2}\right)^2 = \frac{3}{8}W^2$$

- El tiempo del ciclo es $T_{\text{ciclo}} = RTT \cdot W/2$.
- Por lo tanto, el throughput promedio por ciclo es:

$$T = \frac{\text{pkts TX/ciclo}}{\text{tiempo/ciclo}} = \frac{\frac{3}{8}W^2}{RTT \cdot W/2} \quad \text{paquetes/seg}$$

- Por lo tanto, el throughput promedio por ciclo (BW) se define como:

$$BW = \frac{\text{data transmitida por ciclo}}{\text{duración del ciclo}} = \frac{MSS \cdot \text{pkts TX/ciclo}}{T_{\text{ciclo}}} = \frac{MSS \cdot \frac{3}{8} W^2}{RTT \cdot W/2} \quad \text{bits/seg}$$

8.3 Probabilidad de pérdida y paquetes transmitidos

- Sea p la probabilidad de que un paquete sea perdido. La distribución del número de paquetes transmitidos antes de la primera pérdida es:

$$P(\alpha = k) = (1 - p)^{k-1} \cdot p, \quad k = 1, 2, 3, \dots$$

- El valor esperado es:

$$E(\alpha) = \frac{1}{p} \Rightarrow \text{Se transmiten } \frac{1}{p} \text{ paquetes antes de la primera pérdida.}$$

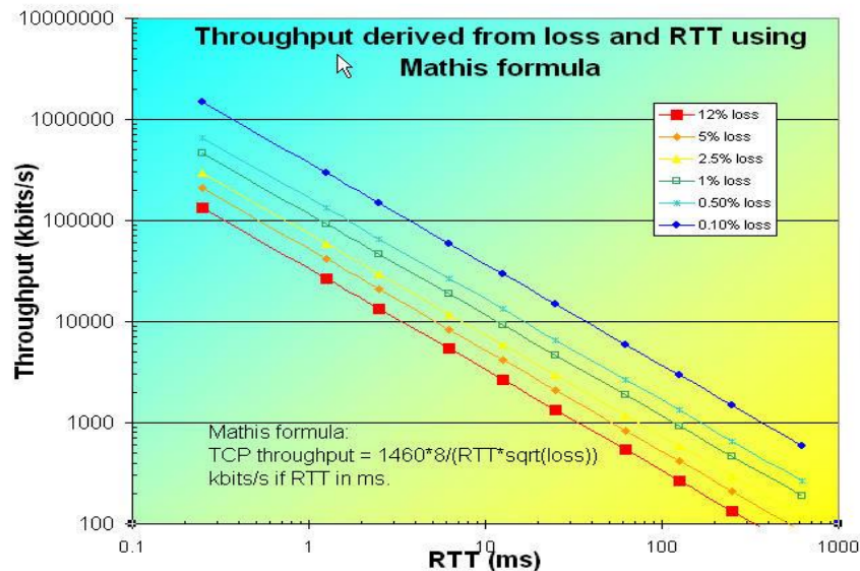
- Incorporando la probabilidad de pérdida p y el número esperado de paquetes transmitidos antes de la primera pérdida ($1/p$), relacionamos con W :

$$W = \sqrt{\frac{8}{3p}}$$

Finalmente, el throughput promedio de TCP en estado estacionario se obtiene como:

$$BW \approx \frac{MSS \times \frac{1}{p}}{RTT \times (\frac{1}{2} \sqrt{\frac{8}{3p}})} = \frac{MSS \times C}{RTT \times \sqrt{p}}$$

8.4 Sender Maximum Segment Size (SMSS) según RFC 2581



- **SMSS:** SMSS es el *tamaño máximo de datos* que un emisor TCP puede enviar en un solo segmento, medido en bytes. Este tamaño **no incluye** los encabezados TCP ni IP.
- **Relación con MTU:** La MTU (*Maximum Transmission Unit*) es el tamaño máximo de un paquete a nivel de enlace que puede enviarse sin fragmentación. SMSS debe ser **menor o igual a la MTU del enlace menos los encabezados TCP e IP**, para evitar fragmentación innecesaria.
- **Path MTU Discovery (RFC 1191):** Cuando los datos atraviesan múltiples enlaces, la MTU puede variar. El Path MTU Discovery permite que TCP determine la MTU efectiva del camino completo entre origen y destino, ajustando el SMSS para evitar fragmentación en ruta.

- **Otros factores que limitan SMSS:**

- RMSS (*Receiver Maximum Segment Size*): límite máximo anunciado por el receptor.
- Configuraciones locales del sistema o restricciones de hardware.
- Políticas de red específicas.